



ARCHITECTURE DECISION RECORD AND FAILURE WALKTHROUGH .



AI DISCLOSURE: NO AI USED FOR THIS ASSIGNMENT

A. Student and project identification

Full Name: KWASHI ELORM FREMA

Student ID: 22397307

Team Name: SEG26-33-MerakiArtsAfrica

Project Title: Meraki Arts Africa Website

Team Repository URL: <https://github.com/Dept-of-Comp-Sci-University-of-Ghana/dcit208-client-project-2026-team-engineering-repository-seg26-33-merakiartsafrica>

Individual Assignment Repository URL: Paste the URL of your GitHub Classroom IA4 repository

B. Decision title and status

ADR-001: Use Separate Frontend and Backend Architecture

Status: Accepted

C. Project Context

The Meraki Arts Africa website allows users to browse programs, view galleries, register for events, donate, and contact the organization. The team decided whether to build everything as one web application or separate the frontend and backend. This decision is important because the website needs to serve many users, securely manage event registrations and donations, and allow both the frontend and backend to be developed independently.

D. Assumptions and constraints

Assumptions

- Users access the site from mobile phones and computers.
- Internet connection may sometimes be unstable.

- The system stores event registrations and contact information.
- Administrators update gallery images and events regularly.

Constraints

- Limited development time.
- Small student development team.
- Free or low-cost deployment.
- Team already knows React, Express and MongoDB.

E. Alternatives

Alternative A (Chosen)

Separate Frontend and Backend

React frontend

Express.js backend

MongoDB database

Frontend communicates using REST APIs

Alternative B (Rejected)

Single Web Application

Everything (HTML, CSS, JavaScript and backend logic) runs in one application.

F. Comparison Matrix

Criterion	Separate Frontend & Backend	Single Web Application
Client requirements	Excellent	Good
Security	Better API protection	Less separation
Maintainability	High	Medium
Scalability	High	Medium
Team collaboration	Excellent	Fair
Deployment flexibility	High	Low
Development speed	Slightly slower initially	Faster initially

G.Decision and Rationale

The team selected the separate frontend and backend architecture because

- React provides a modern and responsive user interface.
- Express manages business logic securely.
- APIs make communication organized.

H. Consequences and Trade-offs

Benefits;

- Easier maintenance
- Better code organization
- Improved security

Disadvantages;

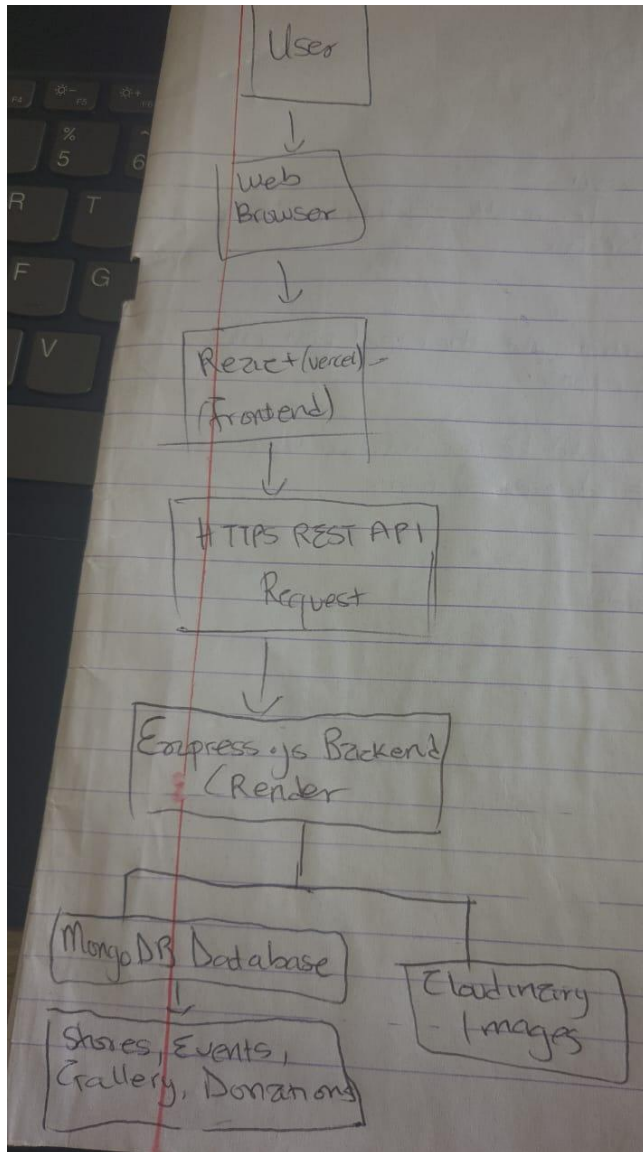
- More API development
- Slightly more complex setup
- Cross-Origin Resource Sharing (CORS) configuration required

Risks;

- Backend downtime prevents frontend from loading data.
- API version mismatches.

When to revisit : If the project becomes very small or maintenance becomes too expensive.

I. Personal architecture sketch



J. Failure Walkthrough

Failure Scenario : A user tries to register for an event, but the backend server is temporarily unavailable.

1. User Action: The user fills out the event registration form and clicks Register.

2. Normal Success Path

- React frontend validates the form.
- The frontend sends a POST request to the Express backend.
- The backend validates the data.
- The backend stores the registration in MongoDB.
- The backend returns a success response.
- The frontend displays "Registration Successful."

3. Failure: The Express backend server is down or unreachable.

4. Failure Detection

- The frontend does not receive a successful response.
- The request times out or returns an HTTP 500 Internal Server Error or 503 Service Unavailable.

5. User Response

Instead of crashing, the website displays:

"Registration could not be completed at the moment. Please try again later."

The user's form data should remain on the page so they do not have to type everything again.

6. Data at Risk

The registration is not saved to MongoDB.

No partial or duplicate registration should be created.

7. Logging

The backend should log:

Date and time

API endpoint (POST /events/register)

Error message

Stack trace

Server status

8. Recovery/Mitigation

- Restore the backend service.
- Retry the request once the server is available.
- Ensure duplicate registrations are prevented.
- Monitor server logs to prevent similar failures.

K. Verification Plan

The team will verify the architectural decision using several methods:

Walking Skeleton

Build a basic end-to-end version of the system where:

React frontend sends a request.

Express backend receives it.

MongoDB stores the data.

A success response is returned.

This confirms that the complete architecture works.

Test Case

Test event registration by:

Entering valid information.

Confirming the backend receives the request.

Confirming the data is stored in MongoDB.

Confirming a success message appears.

Deployment Experiment

Deploy:

React frontend on Vercel.

Express backend on Render.

MongoDB on MongoDB Atlas.

Verify that the deployed frontend communicates correctly with the deployed backend.

Security Check

Verify that:

API endpoints validate user input.

Invalid requests are rejected.

Sensitive data is protected.

CORS is configured correctly.

Client Review

Demonstrate the deployed website to the client and confirm that:

Pages load correctly.

Event registration works.

Gallery images display correctly.

The overall system meets the client's requirements.

L. Oral defence

VIDEO LINK: <https://youtu.be/Dmjz3KhHvo8>

Personal Declaration

Declaration:

I confirm that this submission is based on my team's actual client project. I have identified all AI assistance used. I can explain the decision, alternatives, trade-offs, failure scenario and evidence in my own words.